

Fr. Conceicao Rodrigues College of Engineering

Department of Computer Engineering (CE)

EXPERIMENT 5

Practical No:	5
Title:	Game Playing and Adversarial Search
Date of Performance:	17/3/26
Date of Submission:	17/3/26
Roll No:	10272
Name of the Student:	Sean Pereira

Rubrics for Evaluation:

Sr. No	Performance Indicator	Excellent	Good	Below Average	Total Score
1	On time Completion & Submission (01)	01 (On Time)	NA	00 (Not on Time)	
2	Logic/Theory understanding(02)	02(Correct)	NA	01 (Tried)	
3	Coding Standards (03): Comments/indentation/Naming conventions Output/Test Cases	03(All used)	02 (Partial)	01 (rarely followed)	
4	Post Lab Assignment (04)	04(done well)	3 (Partially Correct)	2(submitted)	

Academic Year	2025-26	Estimated Time	Experiment No. 5 – 02 Hours
Course & Semester	T.E. (CE) – Sem. VI	Subject Name	25PCC13CE17: Artificial Intelligence Lab
Chapter No.	-	Chapter Title	Game Playing and Adversarial Search
Experiment Type	Software/Finding Solution	Software	Prolog/Python

AIM: Write a program to solve a Tic-Tac-Toe game problem in Prolog/ Python.

1. OBJECTIVES

- To gain insights of adversarial search through basic game playing algorithms.
- To be able to implement search space improvement techniques for game playing.

2. DEMONSTRATION OF USEFUL RESOURCES

Game playing assumes multiple-agent environment, and thus offers ideal example for adversarial search. As the agents' goals are in conflict and they always plan against each other, the search space becomes complicated. Moreover, real games involve huge state spaces.

3. Finding Optimal Game Strategies using MINIMAX Algorithm

Two-player board game as a search problem:

- ✓ Players are usually named MAX & MIN. Anyone can start, and they make moves alternating one another.
- ✓ Search problem with 4 components: Initial state, Successor function, Terminal test, Utility function.
- ✓ Strategies of Players: MAX searches for the sequence of moves that leads to a terminal with maximum possible utility value, even if MIN plays in the best way; MIN searches for the opposite, that is, terminal with minimum possible utility.
- ✓ Major steps of the MINIMAX algorithm, from opener's point of view:
 1. Generate the whole game tree.
 2. Find the utility of the terminal nodes.
 3. Determine the MINIMAX values of the non-terminal nodes, from lower nodes up to the root. If a level represents MAX's turn, then the highest values of the successors are taken, and in case of MIN's – lowest values.
 4. Choose the best opening move.

✓ An imaginary game of small depth may be used for explanation. The game of Tic-Tac-Toe is suggested for implementation in Python or Prolog.

- A 3x3 grid is provided with the information of opener, and his/her symbol.
- All nodes up to the terminals are generated, and utilities (-1, 0, +1) are assigned to them.
- The MINIMAX values of non-terminals are computed up to the root, and the winning strategy is returned.

4. Attach the screenshot of the code.

```
% --- WIN CONDITIONS ---
win([0,1,2]). win([3,4,5]). win([6,7,8]).
win([0,3,6]). win([1,4,7]). win([2,5,8]).
win([0,4,8]). win([2,4,6]).

% --- CHECK WINNER ---
winner(Board, Player) :-
    win([A,B,C]),
    nth0(A, Board, Player),
    nth0(B, Board, Player),
    nth0(C, Board, Player).

% --- CHECK DRAW ---
full(Board) :- \+ member(' ', Board).

% --- PRINT BOARD IN TABLE FORMAT ---
print_board([A,B,C,D,E,F,G,H,I]) :-
    write('-----'), nl,
    write(' | '), write(A), write(' | '), write(B), write(' | '), write(C), write(' | '), nl,
    write('-----'), nl,
    write(' | '), write(D), write(' | '), write(E), write(' | '), write(F), write(' | '), nl,
    write('-----'), nl,
    write(' | '), write(G), write(' | '), write(H), write(' | '), write(I), write(' | '), nl,
    write('-----'), nl.

% --- MINIMAX ---
% Base cases
minimax(Board, _, 1) :- winner(Board, 'x').      % AI wins
minimax(Board, _, -1) :- winner(Board, 'o').     % Player wins
minimax(Board, _, 0) :- full(Board).              % Draw

% AI (maximize)
minimax(Board, true, Best) :-
    findall(Score, (
        nth0(I, Board, ' '),
        replace(Board, I, 'x', NewBoard),
        minimax(NewBoard, false, Score)
    ), Scores),
    max_list(Scores, Best).

% Player (minimize)
minimax(Board, false, Best) :-
    findall(Score, (
        nth0(I, Board, ' '),
        replace(Board, I, 'o', NewBoard),
        minimax(NewBoard, true, Score)
    ), Scores),
    min_list(Scores, Best).
```

```

% --- BEST MOVE FOR AI ---
best_move(Board, Move) :-
    findall((Score, I), (
        nth0(I, Board, ' '),
        replace(Board, I, 'x', NewBoard),
        minimax(NewBoard, false, Score)
    ), Moves),
    max_member( (_, Move), Moves).

% --- REPLACE ELEMENT ---
replace(Board, Index, Value, NewBoard) :-
    append(Prefix, [_|Suffix], Board),
    length(Prefix, Index),
    append(Prefix, [Value|Suffix], NewBoard).

% --- START GAME ---
start :-
    play([' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']).

% --- GAME LOOP ---
play(Board) :-
    print_board(Board),

    write('Enter position (0-8): '),
    read(Pos),

    % Player move
    nth0(Pos, Board, ' '),
    replace(Board, Pos, 'o', B1),

    ( winner(B1, 'o') ->
        print_board(B1), write('You win'), nl
    ; full(B1) ->
        print_board(B1), write('Draw'), nl
    ;
        % AI move
        best_move(B1, AI),
        replace(B1, AI, 'x', B2),

        ( winner(B2, 'x') ->
            print_board(B2), write('AI wins'), nl
        ; full(B2) ->
            print_board(B2), write('Draw'), nl
        ; play(B2)
        )
    ).

```

5. Attach the screenshot of the output.

```
101 ?- consult(['c:/Users/Tanay/O
true.

102 ?- start.
-----
|  |  |  |
-----
|  |  |  |
-----
|  |  |  |
-----
Enter position (0-8): 0.
-----
| o |  |  |
-----
|  | x |  |
-----
|  |  |  |
-----
Enter position (0-8): |: 6.
-----
| o |  |  |
-----
| x | x |  |
-----
| o |  |  |
-----

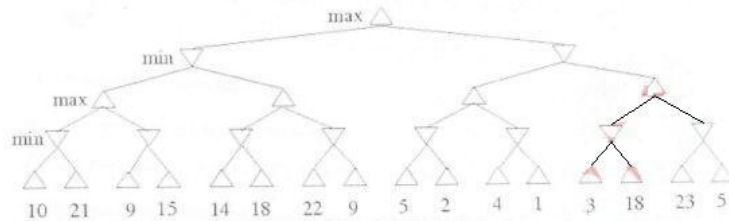
Enter position (0-8): |: 5.
-----
| o |  |  |
-----
| x | x | o |
-----
| o |  | x |
-----
Enter position (0-8): |: 2.
-----
| o | x | o |
-----
| x | x | o |
-----
| o |  | x |
-----
Enter position (0-8): |: 7.
-----
| o | x | o |
-----
| x | x | o |
-----
| o | o | x |
-----
Draw
true .
```

6. Conclusion

The Tic-Tac-Toe game was successfully implemented using SWI-Prolog. The program demonstrates how logical rules and predicates can be used to model a simple game environment. It allows two players to take turns, updates the board after each move, and checks for winning or draw conditions. This experiment helps in understanding game state representation and basic adversarial problem solving in AI. It also shows how Prolog can be used for implementing rule-based game logic.

7. Postlab Questions:

- a. Apply Alpha-Beta pruning on following example considering first node as MAX



- b. Explain MiniMax algorithm with the help of an example

Postlab

a. Alpha-Beta pruning, $\alpha = -\infty$, $\beta = \infty$

b) Minimax is a decision-making algorithm used in two-player games, where one player tries to maximize the score (MAX) and the other tries to minimize the score (MIN). MAX chooses the highest value & MIN chooses the lowest value

Works on recursive tree structure, uses backtracking
 Time complexity: $O(b^d)$ {b=branching factor, d=depth}
 if no. of children = 3, depth = 2 then $T = O(3^2) = O(9)$